AI-DRIVEN SOFTWARE ENGINEERING · FALL 2026 · KBTU SITE

SIS #03 — AI-Assisted Coding, Vibe Coding and Prototype Cleanup

2 points · individual · branch and folder `sis-03` · AI use **Level D** — required; you test, verify, defend and disclose everything you submit. **Deadline: Sunday 1 November 2026, 23:59.**

Reuse what you built. This is Practice #05's `can_book` and Practice #06's Bolt.new page, under the same AC1–AC6. Bring both first versions (v1) in **unchanged** and declare it; missed a practice? Generate that v1 fresh. What is new is the evidence: one test table for both, traceability, an issue log, the AI's critique checked claim by claim, the agent loop, a reflection.

Your choices: the coding assistant (record the exact model — "ChatGPT" is a tool) and the language of the function — Python recommended (Path A, the checker runs it), any other is Path B and loses nothing. The page is **Bolt.new** for everyone.

Full instructions: `sis-03/README.md` in `SIS_03.zip`. **Time:** ~4–5 h reusing, ~6–7 h fresh, over two days or more.

The system — from the brief, unchanged

FIXED - REJECT AI SUGGESTIONS THAT CHANGE THEIR MEANING

```
User story: As a student, I want to check a room's availability so that I can choose a valid
booking time.
One responsive page, one selected room, fictional bookings. Checks availability only - never
creates or confirms.
Excluded: login, database, backend, external API, payments, unrelated features.
AC1  integer minutes: 0 <= start < end <= 1440; start strictly after now
AC2  duration at most 120 minutes; exactly 120 is permitted
AC3  reject every request when the room is blocked
AC4  reject overlap with any booking; touching at an endpoint is NOT overlap
AC5  an unambiguous Available / Unavailable; never mutate existing bookings
AC6  understandable validation feedback; usable narrow/mobile layout
Contract: can_book(start, end, now, blocked, existing) -> True only when AC1-AC4 hold
```

The work, in order

Step	You do	report.md
0 Baseline	Scope, AC1–AC6, two failure points, contract, assumptions — your own words. Commit and push FIRST.	§2
A Plan · B Generate	AI plan you approve; Python v1 saved unedited in code/original/; Bolt v1 downloaded into web/original/ with web-v1.png. Review every branch of v1.	§3, §4, §11
Test	T1–T12 on both artifacts, v1 and final. Python: what the call returned. Page: what it showed. 10+ tests in test_booking.py.	§5, §6, §12
C Critique	AI review against AC1–AC6; verify each claim; accept, reject or modify — one at least rejected or modified.	§8
D Repair	2+ issues: evidence → focused prompt → change → retest. before-1 / after-1 screenshots. Final into web/final/, final-narrow.png at phone width.	§7
E Verify · Write	Rerun everything. Agent loop, observed vs assumed, a safety checkpoint. Reflection 100–200 words.	§9, §10, §13

The twelve cases

Base for every case: `now = 540, blocked = False, existing = [(600, 660)]`.

Case	Input	Expected	Case	Input	Expected
T1 Touching	660–720	Available	T7 Reversed	720–660	Unavailable
T2 Overlap	630–690	Unavailable	T8 Ends at midnight	1380–1440	Available
T3 Blocked	660–720, blocked	Unavailable	T9 Past midnight	1380–1441	Unavailable
T4 Exactly 120	720–840	Available	T10 Second booking	720–780 + (700–760)	Unavailable
T5 121 min	720–841	Unavailable	T11 Same check twice	660–720, twice	Available
T6 Starts now	540–570	Unavailable	T12 Narrow screen	T1 at ~375 px	Available

Never claim a test passed unless it was actually executed. On Path A the checker runs both of your Python files on T1–T11 and compares them with what you wrote. On the page, T8 and T9 may be Not testable (a time input stops at 23:59).

Check, declare, submit

```
cd sis-03
python tests/check_sis.py            # 55 checks on Path A, 36 on Path B
git add . && git commit -m "sis-03: final versions and report"
git rev-parse --short HEAD           # into submission.yml checker.commit
python tests/validate_submission.py
git add submission.yml && git commit -m "sis-03: declaration" && git push
```

1. **Branch** `sis-03`, folder `sis-03/`, 3+ meaningful commits — the first is your baseline.
2. Pull request `sis-03 → main` in your own repository, five-section description from SETUP.md. **Leave it open.**
3. In Teams: **+ Add work → Link → the pull request URL**. Not an e-mail, not a file, not the repository URL.

Grading — 2 points, 8 × 0.25

Criterion	Full 0.25 when …
Baseline & contract	Your own words, two real failure points, committed before the first new AI exchange.
Function & tests	AC1–AC5 hold; v1 kept and reviewed branch by branch; 10+ green tests; surviving faults caught or explained.
Prototype & cleanup	Prompt with AC1–AC6 and the exclusions; v1 and final exported; final differs, in scope, usable at phone width.
Test execution & traceability	Twelve cases, both artifacts, v1 and final, actually observed; AC1–AC6 traced; real suite output.
Issues & critique	2+ issues with focused prompts and retests, before/after; three claims verified, one rejected or modified.
Coding-agent analysis	A real pass through the loop; observed and assumed kept apart; a concrete checkpoint; dependability argued.
Reflection	100–200 words, your own: comparison, three risks, responsibility, a rejected recommendation, a next step in scope.
AI evidence & workflow	4+ exchanges (Plan, Generate, Critique, Repair); reuse declared; open PR; submission.yml matches the re-run.

A FAIL you report and explain costs you nothing. One you hide costs the whole criterion.

Worthless: “the AI was faster and I fixed some bugs.” Worth everything: “T1 returned False on v1 — start <= b rejects a booking that starts when the other ends; AC4 allows it.”